

User-Space Application Documentation for Interacting with /dev/my_ioctl_device

This document describes a user-space application that interacts with the /dev/my_ioctl_device character device to exchange strings with the kernel module.

1. Header Files

The application requires the following standard C libraries:

- `<stdio.h>`: Provides standard input/output functions like `printf`.
- `<stdlib.h>`: Provides general utility functions like `perror`.
- `<fcntl.h>`: Provides file control constants like `O_RDWR`.
- `<unistd.h>`: Provides various POSIX operating system functions like `open` and `close`.
- `<sys/ioctl.h>`: Provides functions for performing IOCTL operations on devices.

2. Macros

- `DEVICE_PATH`: Defines the path to the character device (`/dev/my_ioctl_device`).
- `IOCTL_SET_STRING`: Macro representing the IOCTL command for setting a string to the kernel (`_IOW('k', 1, char *)`).
- `IOCTL_GET_STRING`: Macro representing the IOCTL command for getting a string from the kernel (`_IOR('k', 2, char *)`).

3. Global Variables

- `fd`: File descriptor for the character device.
- `user_string`: Character array to store the string to be sent to the kernel (size 100 bytes).
- `kernel_response`: Character array to store the received string from the kernel (size 100 bytes).

4. Program Flow

1. Open the device:

- `open(DEVICE_PATH, O_RDWR)` attempts to open the character device `/dev/my_ioctl_device` with read-write access.
- On success, a file descriptor (`fd`) is returned for interaction with the device.
- On failure, `perror` is called to print the system error message, and the program exits with `EXIT_FAILURE`.

2. Set the string to the kernel:

- `snprintf(user_string, sizeof(user_string), "Hello from user space!")` formats the string "Hello from user space!" into the `user_string` buffer with size checking.
- `ioctl(fd, IOCTL_SET_STRING, user_string)` performs an IOCTL operation on the device using the file descriptor (`fd`). The specific command is `IOCTL_SET_STRING`, and the argument is the pointer to the `user_string` buffer containing the data to be sent.
- On success, the `ioctl` call returns 0. On failure, `perror` is called to print the system error message, the device is closed, and the program exits with `EXIT_FAILURE`.

3. Get the string from the kernel:

- `ioctl(fd, IOCTL_GET_STRING, kernel_response)` performs another IOCTL operation on

the device. This time, the command is `IOCTL_GET_STRING`, and the argument is a pointer to the `kernel_response` buffer where the received data will be stored.

- On success, the `ioctl` call returns 0. On failure, `perror` is called to print the system error message, the device is closed, and the program exits with `EXIT_FAILURE`.

4. Print the received string:

- `printf("String received from the kernel: %s\n", kernel_response)` prints the string received from the kernel stored in the `kernel_response` buffer.

5. Close the device:

- `close(fd)` closes the file descriptor associated with the device, releasing resources.

6. Exit the program:

- The program exits with `EXIT_SUCCESS` indicating successful execution.